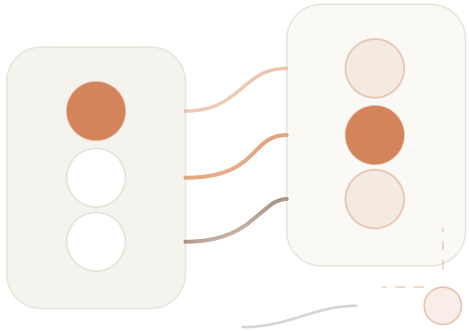
G4 WEEK 7 · PRE-SEARCH

ChatBridge

Building an AI chat platform with third-party app integration

Analysis | Decisions | System design | Auth | Chess E2E | Connections | Risks | Why it wins | Wireframes | Sources



REQUIRED PRE-SEARCH SECTION

Case Study Analysis

TutorMeAI's core challenge is turning a chat product into a platform — one where third-party apps run inside the conversation while preserving the safety guarantees that [redacted]900 school districts depend on daily. The moment outside code renders UI and [redacted]ed by children, every design decision becomes a trust [redacted]

The first key problem is the [redacted]

The third problem is **authentication across trust boundaries**. Some apps require user-level credentials, like a Spotify integration that creates playlists on behalf of a student. Letting the embedded app run its own OAuth flow inside an iframe introduces phishing risk, malicious redirects, and insecure token storage. We chose platform-mediated authentication: the chat platform handles OAuth consent, stores and refreshes tokens, and passes only scoped capabilities to the app at invocation time. The ethical weight here is significant — these are children's accounts. The platform must be the single point of credential trust.

The ethical dimension runs through every layer. Content moderation for apps used by minors is a first-class concern. We chose a reviewed allowlist model rather than an open marketplace — every app must be approved before appearing in any classroom. App descriptions and capabilities are moderated. Emergency kill switches let administrators disable an app instantly across all districts. This is more restrictive than an open ecosystem, but for K-12 education the cost of a harmful app reaching students far exceeds the cost of slower developer onboarding.

What we landed on is a manifest-based plugin architecture with sandboxed rendering, structured event communication, platform-owned auth, and an admin-reviewed app registry. It prioritizes defensibility over openness — the right posture for a product whose users are children and whose customers are school districts that expect zero safety incidents.

ARCHITECTURE DECISIONS

Decision matrix

Three approaches per phase. B is the pick.

A

CONSTRAINTS
Demo-first
Bundled apps, weak trust

ARCHITECTURE
Local plugin
Shared runtime, in-process

HARDENING
Minimal
Basic retries, manual list

B ✓

CONSTRAINTS
Reviewed allowlist
Per-origin isolation, scoped

ARCHITECTURE
Iframe SDK
postMessage bridge, BFF auth

HARDENING
Zero-trust
Scoped perms, audit logs

C

CONSTRAINTS

Marketplace

Ecosystem, too much scope

ARCHITECTURE

MCP protocol

Remote servers, high build cost

HARDENING

Platform-grade

Policy engine, overbuilt

BOUNDARY

Sandboxed iframe + postMessage

AUTH

Platform-mediated OAuth via BFF

STATE

App snapshots → platform context

SECURITY

Reviewed apps, scoped perms, kill switches

RECOMMENDED ARCHITECTURE

System design

Messaging and history already exist in the product. This section covers the missing requirements only: app context, multi-turn continuity, tool registration, tool schemas, embedded UI, structured invocation, completion signaling, and independent app state.

Idea 1 · Fastest

Local plugin contract

- Apps register capabilities directly with the host product.
- Tool schemas live in the same platform runtime, so discovery is simple.
- UI renders as trusted internal components inside the chat area.
- Multi-turn context is easiest because app state and chat state share one store.
- Focus: optimized for speed and a tightly coupled host boundary.

Best for: fastest prototype, lowest realism.

Idea 2 · Recommended

Manifest + iframe bridge

- Each app registers through a manifest: id, capabilities, tool schemas, widget URL, auth type, allowed origins.
- The model discovers tools from manifest-derived schemas and invokes them with structured parameters.
- The UI renders inside the chat as a sandboxed iframe, so the app can own its own interface and state.
- Multi-turn works through state snapshots sent back to the platform after each meaningful user action.
- The app signals `complete`, `error`, and `state.update`, so the chat stays aware while the app retains its internals.

Best for: strongest fit to the assignment.

3 · Most extensible

te tool server + widget shell

- os expose capabilities through a remote tool server, while UI comes from a arate widget endpoint.
- l discovery is clean and protocol-shaped, good for future ecosystem growth.
- lti-turn context still depends on structured state echo from the app back to the tform.
- mpletion and error signals are explicit lifecycle events driven by the app contract.
- ak point: bigger build cost and more integration pieces this week.

or: long-term platform story, highest complexity.

SEARCH FINDING 1

I boundary

I Apps SDK patterns, MDN iframe sandbox guidance, and OWASP third-party JS ce all point toward isolated embedded UI with a dedicated host boundary.

ets the app render rich UI in chat while keeping the host product in control.

SEARCH FINDING 2

ommunication model

`postMessage` plus OAuth browser-app guidance support a narrow, origin-checked ge bridge with explicit event types.

he chatbot gets structured app state instead of guessing from visible UI.

SEARCH FINDING 3

uth model

00 and browser-app OAuth guidance favor platform-mediated auth with PKCE and token

Why: a

session

Architecture

```
Chat App
├── Conversation UI
├── Tool Router
├── App Registry
├── Widget Host (sandboxed iframe)
├── App Context Store
├── Policy Guard
└── OAuth Broker

Third-Party App
├── manifest.json
├── tool schemas
├── widget UI
└── internal state
```

Host owns trust, routing, and conversation context. App owns UI and its own working state.

Signal chain

```
User asks for task
  → model selects tool from registered schemas
    → platform validates app + scope + auth
      → widget mounts inline in chat
        → structured invoke sent to app
          → app emits state.update snapshots
            → snapshots appended to chat context
              → user asks follow-up
                → model answers from chat + app state
                  → app emits complete or error
                    → chat continues naturally
```

Key rule: published state becomes model context through the structured bridge.

Data model

```
conversation/
  {conversation_id}/
    messages/
    active_apps/
      {app_instance_id}/
        app_id
        status
        approved_scopes.json
        last_tool_call.json
        latest_state_snapshot.json
        completion.json

app_registry/
  {app_id}/manifest.json
  {app_id}/policy.json

oauth_connections/
  {user_id}/{app_id}.json
```

Conversation holds only the app state needed for future turns. Full internal app state stays with the app.

What you are actually building

```
1. App registration contract
2. Tool schema discovery layer
3. Inline widget host in chat
4. Structured invoke/event bridge
5. App snapshot → model context path
6. Completion/error lifecycle
7. Platform-mediated auth for protected apps
```

That is the minimum technical slice needed to satisfy the missing requirements.

Recommended contract

Register

App submits a manifest with capabilities, auth pattern, widget endpoint, and discoverable tool schemas.

Discover

The platform turns those schemas into callable tools the chatbot can choose during a conversation.

Render

When selected, the app UI mounts inline in the chat experience as part of the same conversation surface.

Invoke

The chatbot sends structured parameters to the app through explicit typed inputs.

Signal

The app returns lifecycle events: `state.update`, `complete`, and `error`.

State

The app keeps its own internal state, while the platform stores summarized state needed for future turns.

How multi-turn context works

Turn 1

User asks for an app task. The chatbot chooses a tool and mounts the matching app UI.

Turn 2

User interacts with the app. The app publishes a structured snapshot such as board state, playlist draft, or selected options.

Turn 3

User asks a follow-up like “what should I do here?” The chatbot answers using chat history plus the latest app snapshot.

Turn 4

The app emits completion. The chatbot treats the result as conversation context and continues naturally after the tool finishes.

Rule

The platform uses published app state as the source of truth for chatbot understanding.

AUTH BY APP TYPE

Authentication matrix

Auth flow by app category

App Type	Auth Pattern	Token Owner	Example
Internal	None	Platform	Chess, calculator
External Public	API key (server)	Platform	Weather, dictionary
External Auth	OAuth2 + PKCE	Platform BFF	Spotify, GitHub

The platform always owns credentials. Apps receive scoped capabilities per session.

OAuth flow for authenticated apps

```
User clicks "Connect Spotify"
  → platform opens OAuth consent in popup (owned origin)
    → user approves scopes on provider site
      → provider redirects to platform callback
        → platform stores encrypted refresh token
          → platform mints scoped session token
            → session token passed to app via bridge
              → app calls provider API with session token
                → platform auto-refreshes on expiry
```

The app widget never handles raw refresh tokens or redirect flows.

REFERENCE IMPLEMENTATION

Chess: end-to-end sequence

Full lifecycle

```
1. User: "let's play chess"
2. Model: selects chess.start_game tool
3. Platform: validates chess app → approved, internal, scope ok
4. Platform: mounts chess widget iframe inline in chat
5. Platform: sends invoke { tool: "start_game", params: { color: "white"
} }
6. Chess app: initializes board, emits state.update
   → { fen: "rnbqkbnr/...", turn: "w", move_count: 0 }
7. Platform: appends snapshot to conversation context
8. User: drags e2 → e4 on board UI
9. Chess app: validates move, updates board, emits state.update
   → { fen: "rnbqkbnr/.../P.../...", turn: "b", last_move: "e2e4" }
10. User: "what should I do here?"
11. Model: reads chat + latest chess snapshot → suggests strategy
12. User: plays several more moves via board UI
13. Chess app: detects checkmate, emits complete
    → { result: "checkmate", winner: "white", moves: 24 }
14. Platform: closes widget, adds result to conversation context
15. Model: "Great game! You won in 24 moves. Want to analyze it?"
```

Every step maps to a specific contract primitive: invoke, state.update, complete.

Chess manifest

```
{
  "id": "chess",
  "name": "Chess",
  "version": "1.0.0",
  "description": "Interactive chess with move validation",
  "auth": { "type": "none" },
  "widget": {
    "url": "https://apps.chatbridge.dev/chess",
    "sandbox": ["allow-scripts"],
    "dimensions": { "width": 400, "height": 440 }
  },
  "tools": [
    {
      "name": "start_game",
      "description": "Start a new chess game",
      "parameters": {
        "type": "object",
        "properties": {
          "color": { "type": "string", "enum": ["white","black"] }
        }
      }
    },
    {
      "name": "get_board_state",
      "description": "Get the current board position and game status",
      "parameters": { "type": "object", "properties": {} }
    },
    {
      "name": "suggest_move",
      "description": "Analyze the board and suggest the best move",
      "parameters": { "type": "object", "properties": {} }
    }
  ],
  "events": ["state.update", "complete", "error"],
  "scopes": ["chat.context.read"]
}
```

This is the contract a third-party developer submits to register with the platform.

REQUIREMENTS TO PRODUCT BOUNDARY

How it connects to the app

Only the required connection points: how third-party apps plug into the chat product and what crosses the boundary.

1. Chat surface

The app keeps its existing chat experience, but adds one new surface: an embedded app panel inside the conversation.

When the assistant selects a tool, the panel opens inline as part of the same conversation surface.

The chat transcript also shows app lifecycle messages: launched, waiting, connected, completed, and recovery states.

2. App host boundary

The host product loads the third-party UI in a sandboxed iframe and passes only minimal startup data: app id, session id, theme, and approved scopes.

All communication goes through a narrow event bridge: invoke tool, send state snapshot, emit completion, emit error.

The embedded app operates through a scoped bridge into the broader chat UI and approved platform APIs.

3. Platform services

The app needs a registry service for app manifests, a policy layer for safety checks, and a state service that stores app context alongside conversation context.

That is what lets the assistant answer follow-up questions like “what should I do here?” after the user has already interacted with the widget.

The model reads structured state published back through the contract as the primary app-context channel.

4. Auth and trust path

For authenticated apps, the user authorizes through the platform in a platform-owned consent flow.

The platform stores and refreshes tokens, then gives the app only the minimum approved capability needed for that session.

This is the key connection between the requirements and the app: the chat remains the trusted control plane, while third-party apps stay isolated workers.

PRESENTATION ASSETS

Wireframes

Two high-signal frames for the architecture walkthrough: one stateful app flow, one authenticated app flow.

WIREFRAME 1

Chat with active chess app

Narrative: user starts chess, board mounts inline, assistant stays context aware.

WIREFRAME 2

OAuth connect flow for Spotify

Narrative: platform handles auth, app receives only scoped capability after consent.

ADJACENT PROBLEMS

Risks that still need answers

These sit next to the core plugin problem and shape whether districts trust the platform enough to buy and expand it.

01

District policy controls

Each district will want its own app allowlist, scope rules, moderation posture, and emergency disable controls.

02

Student privacy records

App snapshots, auth events, and tool outputs may become part of regulated student records and need retention rules.

03

Prompt injection path

Tool descriptions, app metadata, and state payloads can shape model behavior and need validation plus policy filtering.

04

Embed compatibility

Many third-party products ship CSP and frame policies that shape which apps qualify for the platform contract.

05

Accessibility quality

Embedded apps need keyboard support, readable semantics, and classroom-grade accessibility to fit district procurement.

06

Cost and observability

State snapshots, repeated tool loops, and multi-turn analysis increase model cost and raise the need for strong tracing.

BUSINESS IMPORTANCE

Why this matters and why TutorMeAI wins

The strategic value comes from becoming the trusted orchestration layer between student chat, teacher control, and approved classroom tools.

Why this matters

Product shift

The product moves from answering questions to orchestrating classroom workflows inside one conversation surface.

Retention

Once chat history, app state, teacher settings, and district policies live together, switching costs rise sharply.

Trust

K-12 buyers reward platforms that combine utility with privacy, moderation, auditability, and strong policy controls.

Expansion

A safe app layer creates room for more subjects, more districts, and deeper teacher workflows over time.

Why TutorMeAI wins

Current edge

The case study already says teachers can shape chatbot behavior, tone, and focus better than competitors.

Next edge

That configurability extends naturally into approved app selection, app routing, and context-aware orchestration.

Hard to copy

Competitors can copy chat UI quickly; district trust, policy controls, reviewed integrations, and context-linked app state take longer.

Winning thesis

TutorMeAI wins by becoming the trusted operating layer between classroom chat, district guardrails, and third-party learning tools.

WEB RESEARCH

Research sources used

Grouped for fast reading: security boundary, privacy policy, interoperability, and educa
adoption context.

Security · OpenAI Apps SDK Reference

developers.openai.com/apps-sdk/reference

Useful for tool descriptors, widget metadata, dedicated component origins, and JSON-RPC style UI bridge patterns.

Security · OpenAI Apps Security & Privacy

developers.openai.com/apps-sdk/guides/security-privacy

Supports the case for sandboxed iframes, strict CSP, and limiting widget capabilities by default.

Security · MDN postMessage

developer.mozilla.org/.../Window/postMessage

Grounds secure cross-origin messaging. Critical point: always validate sender and target origins.

Security · MDN iframe sandbox

developer.mozilla.org/.../HTMLIFrameElement/sandbox

Supports app isolation by selectively granting only the smallest needed iframe capabilities.

Security · MDN CSP directives

developer.mozilla.org/.../CSP/frame-ancestors

developer.mozilla.org/.../CSP/sandbox

Explains both who may embed a page and what a sandboxed page is allowed to do once embedded.

S ity · RFC 9700 + Browser Apps

r itor.org/rfc/rfc9700.html

d racker.ietf.org/.../draft-ietf-oauth-browser-based-apps

S rts PKCE, secure in-browser messaging checks, and the choice to keep tokens away f ntrusted widget code.

I perability · MCP tools spec

m ontextprotocol.io/specification/2025-06-18/server/tools

U reference for tool discovery semantics and structured tool metadata design.

S ity · OWASP third-party JS

d heetseries.owasp.org/.../Third_Party_Javascript_Management_Cheat_Sheet.html

Reinforces iframe sandboxing, CSP hardening, and the general danger of giving third-party code to

Education policy · UNESCO GenAI guidance

unesco.org/en/articles/guidance-generative-ai-education-and-research

Useful for the importance of governance, safe adoption, and institution-level controls around AI in education.

Privacy policy · FERPA

studentprivacy.ed.gov/ferpa

Grounds why conversation state, app outputs, and student-linked records need strong privacy and retention design.

Privacy policy · COPPA

ftc.gov/legal-library/browse/rules/childrens-online-privacy-protection-rule-coppa

Important for any platform used by children where third-party tools may touch identifiers, content, or account-linked behavior.

Interoperability · 1EdTech LTI

`1edtech.org/standards/lti`

Useful reading for why interoperability matters in education and how platform standards shape long-term integration strategy.

Education policy · U.S. AI toolkit

`files.eric.ed.gov/fulltext/ED661924.pdf`

Helpful for district-facing implementation framing: policy, governance, safety, and operational readiness.

Presentation summary

Strong live summary: sandboxed iframe boundary, origin-checked state bridge, platform-mediated OAuth, district-grade privacy posture.